

LAB 1.2: Incremental Encoder

สมาชิก

- นาย ตะวัน นพเกตุ 67340500014
- นาย นายพืระกานต์ ยู 67340500031
- นาย นายพุดิพงษ์ วงษ์พงษ์ 67340500073

จุดประสงค์ (Objectives)

1. เพื่อศึกษาและแสดงลักษณะของสัญญาณดิบ (Quadrature Signals) ที่อ่านได้จากช่อง A และ B และพิสูจน์ความสัมพันธ์เชิงเฟส (Phase Relationship) เมื่อหมุนตามเข็ม (CW) และทวนเข็ม (CCW)
2. เพื่อศึกษาปัญหา Overflow (การวนลูป) ของ Hardware Counter 16-bit และออกแบบอัลกอริทึม Wrap-around เพื่อแก้ไขปัญหา ทำให้สามารถวัดตำแหน่งสะสม (Accumulated Count) ได้ต่อเนื่อง
3. เพื่อทดลองวัดและคำนวณหาค่า Pulses Per Revolution (PPR) ของ Encoder ที่ใช้ และคำนวณหาค่าความละเอียดเชิงมุม (Angular Resolution) ของระบบ
4. เพื่อแปลงค่า Raw Counts ที่ผ่านการแก้ไข Overflow แล้ว ให้เป็นหน่วยทางวิศวกรรม ได้แก่ ตำแหน่งเชิงมุม (Radians) และความเร็วเชิงมุม (rad/s)
5. เพื่อออกแบบและนำฟังก์ชัน Homing Sequence (การตั้งศูนย์) ไปใช้งาน
6. เพื่อวิเคราะห์และเปรียบเทียบผลลัพธ์ของโหมดการถอดรหัส (Decoding Modes) แบบ X1, X2 และ X4
7. เพื่อวิเคราะห์ผลกระทบของความเร็วในการหมุนที่มีต่อคุณภาพของสัญญาณความเร็วเชิงมุม

หลักการงานและทฤษฎีที่เกี่ยวข้อง

การอ่านค่าจาก Incremental Encoder อาศัยหลักการหลายส่วนร่วมกันเพื่อแปลงการหมุนเชิงกลให้เป็นข้อมูลดิจิทัลที่สามารถประมวลผลได้ ดังนี้

1. สัญญาณ Quadrature (A และ B)

Incremental Encoder สร้างสัญญาณเอาต์พุตเป็นคลื่นสี่เหลี่ยม (Square Wave) 2 ช่องสัญญาณ คือ A และ B สัญญาณทั้งสองนี้ถูกออกแบบมาให้มีเฟสต่างกัน 90 องศา

- **การตรวจจับทิศทาง:** หลักการพื้นฐานของ Quadrature คือการตรวจจับทิศทางการหมุน ไมโครคอนโทรลเลอร์สามารถตรวจสอบสถานะของสัญญาณหนึ่ง ในขณะที่ขอบสัญญาณ (เช่น ขอบขาขึ้น) ของอีกสัญญาณหนึ่งเกิดขึ้น
 - **หมุนตามเข็มนาฬิกา (CW):** สัญญาณ B จะนำหน้า (Leads) สัญญาณ A
 - **หมุนทวนเข็มนาฬิกา (CCW):** สัญญาณ A จะนำหน้า (Leads) สัญญาณ B
- **การวัดแบบสัมพัทธ์ (Relative):** Encoder ประเภทนี้ไม่ได้บอก ตำแหน่งสัมบูรณ์ แต่บอก การเปลี่ยนแปลง ของตำแหน่ง เมื่อหยุดหมุน สัญญาณจะค้างอยู่ที่สถานะลอจิกสุดท้าย (เช่น A=1, B=0) จึงจำเป็นต้องมีกระบวนการ Homing เพื่อตั้งค่าจุดอ้างอิงเริ่มต้น (ศูนย์) ด้วยซอฟต์แวร์

2. โหมดการถอดรหัส (Decoding Modes)

คือวิธีการนับพัลส์จากสัญญาณ A และ B เพื่อให้ได้ความละเอียดที่แตกต่างกัน:

- **X1 (Single-edge counting):** เป็นวิธีพื้นฐานที่สุด โดยจะนับเฉพาะ ขอบขาขึ้น ของสัญญาณ A เท่านั้น ทำให้ได้จำนวนพัลส์ต่อรอบเท่ากับค่า PPR
- **X2 (Double-edge counting):** จะนับทั้ง ขอบขาขึ้นและขอบขาลง ของสัญญาณช่องเดียว ทำให้ได้ความละเอียดเป็น 2 เท่าของ PPR
- **X4 (Quadrature decoding):** เป็นโหมดที่ให้ความละเอียดสูงสุด โดยฮาร์ดแวร์จะนับ ทุก ขอบ (ทั้งขาขึ้นและขาลง) ของทั้งสองสัญญาณ (A และ B) ทำให้ได้ความละเอียดเป็น 4 เท่าของ PPR

3. ปัญหา Overflow และ อัลกอริทึม Wrap-around

ในการทดลองนี้ Hardware Timer ที่ใช้เป็นแบบ 16-bit ซึ่งหมายความว่าตัวนับ (Counter Register) สามารถนับค่าได้สูงสุดเพียง $2^{16} - 1 = 65535$

- **Overflow:** เมื่อตัวนับนับถึง 65535 ในพัลส์ถัดไป ค่าจะวนกลับ (Rolls over) มาเริ่มต้นที่ 0 ใหม่
- **Underflow:** ในทางกลับกัน หากนับถอยหลังจาก 0 ค่าจะวนกลับไปเป็น 65535
- **Wrap-around:** คืออัลกอริทึมซอฟต์แวร์ที่ออกแบบมาเพื่อตรวจสอบการกระโดดของค่าที่ผิดปกติ (เช่น จาก 65534 ไป 1 หรือ 1 ไป 65534) และทำการชดเชยค่าทางคณิตศาสตร์ (เช่น บวกหรือลบด้วย 65536) เพื่อให้ได้ค่าตำแหน่งสะสม (Accumulated Count) ที่ต่อเนื่อง

การตั้งค่าฮาร์ดแวร์ (IOC Setup)

เพื่อให้สามารถทดลองและเปรียบเทียบโหมดการถอดรหัสที่แตกต่างกันได้ จึงมีการตั้งค่า Timer 3 ตัวในซอฟต์แวร์ STM32CubeIOC ดังนี้:

1. TIM3 (สำหรับโหมด X4):

- **เหตุผล:** เลือกใช้ TIM3 ซึ่งเป็น General-Purpose Timer ที่รองรับ Encoder Mode
- **การตั้งค่า:** เลือกโหมด Encoder Mode TI1 and TI2 เป็นการสั่งให้ฮาร์ดแวร์ใช้ทั้งช่องสัญญาณ TI1 (PA4) และ TI2 (PA6) เพื่อนับทุกขอบของสัญญาณ A และ B (X4 Decoding)
- **Counter Period (ARR):** ตั้งค่าไว้ที่ 65535 ซึ่งเป็นค่าสูงสุดของ 16-bit เพื่อให้มีช่วงการนับที่กว้างที่สุดก่อนเกิด Overflow

2. TIM1 (สำหรับโหมด X2):

- **เหตุผล:** เลือกใช้ TIM1 ซึ่งเป็น Advanced-Control Timer
- **การตั้งค่า:** เลือกโหมด Encoder Mode TI1 ฮาร์ดแวร์จะนับทั้งขอบขาขึ้นและขาลงของสัญญาณที่ช่อง TI1 (PA8) เท่านั้น ส่วน TI2 (PC1) จะใช้เพื่อกำหนดทิศทาง ซึ่งให้ความละเอียดแบบ X2

3. TIM4 (สำหรับโหมด X1):

- เหตุผล: เลือกใช้ TIM4 ซึ่งเป็น General-Purpose Timer
- การตั้งค่า: เลือกโหมด Encoder Mode TI2 ซึ่งทำงานกลับกันกับ TIM1 คือ ฮาร์ดแวร์จะนับเฉพาะขอบขาขึ้นของสัญญาณที่ช่อง TI2 (PA12) เท่านั้น ส่วน TI1 (PA11) ใช้กำหนดทิศทาง ซึ่งให้ความละเอียดแบบ X1

การทดลองที่ 1: การอ่านสัญญาณดิบและทิศทางการหมุน

1. จุดประสงค์

เพื่อศึกษาสัญญาณดิบ A/B และพิสูจน์ความสัมพันธ์เชิงเฟส (Phase Relationship) เมื่อหมุน CW และ CCW

2. สมมติฐาน

สัญญาณ A และ B จะเป็นคลื่นสี่เหลี่ยมที่มีเฟสต่างกัน 90 องศา และลำดับการนำหน้า (Lead/Lag) ของสัญญาณจะสลับกันเมื่อเปลี่ยนทิศทางการหมุน

3. ตัวแปร

- ตัวแปรต้น: ทิศทางการหมุน (หยุดนิ่ง, CW, CCW)
- ตัวแปรตาม: สถานะลอจิกของสัญญาณ A และ สัญญาณ B
- ตัวแปรควบคุม: Sample Time (0.001 s)

4. ขั้นตอนการดำเนินงาน

1. สร้างแบบจำลอง Simulink โดยใช้บล็อก Digital Port Read 2 ตัว เพื่ออ่านค่าจากขา PA4 (สัญญาณ A) และ PA6 (สัญญาณ B) (ดูรูปที่ 1 ในภาคผนวก)
2. ตั้งค่า Sample Time เป็น 0.001 s
3. ทดลองหมุน Encoder อย่างช้า ๆ ทั้งสองทิศทาง และสังเกตผลด้วยบล็อก Scope ใน Simulink

5. ผลการทดลอง (ดูกราฟที่ 1, 2, 3 ในภาคผนวก)

- **ขณะหยุดนิ่ง (ภาพที่ 1):** สัญญาณ A และ B จะคงสถานะลอจิกสุดท้ายไว้ (ในภาพคือ $A=Low$, $B=Low$)
- **ขณะหมุนตามเข็มนาฬิกา (CW) (ภาพที่ 2):** สังเกตเห็นว่า สัญญาณ B (สีม่วง) นำหน้า (Leads) สัญญาณ A (สีเขียว)
- **ขณะหมุนทวนเข็มนาฬิกา (CCW) (ภาพที่ 3):** ความสัมพันธ์กลับด้าน โดย สัญญาณ A (สีเขียว) นำหน้า (Leads) สัญญาณ B (สีม่วง)

6. สรุปผลการทดลอง

ผลการทดลองยืนยันสมมติฐานว่า Incremental Encoder ใช้หลักการ Quadrature (เฟสต่างกัน 90 องศา) เพื่อตรวจจับทิศทางการหมุน โดย CW และ CCW ให้ลำดับการนำหน้าของสัญญาณ A และ B ที่สลับกัน

7. อภิปรายผล

- ข้อสังเกตสำคัญคือเมื่อหยุดหมุน สัญญาณไม่ได้กลับไป 0 เสมอไป แต่จะค้างสถานะสุดท้ายไว้ตามตำแหน่งที่จานหมุนบังแสงอยู่
- นี่คือพฤติกรรมปกติของ Incremental Encoder ซึ่งวัด การเปลี่ยนแปลง (Relative) ไม่ใช่ ตำแหน่งสัมบูรณ์ (Absolute)
- พฤติกรรมนี้ยืนยันว่าการใช้งาน Encoder ประเภทนี้จำเป็นต้องมีกระบวนการ "Homing" (การตั้งศูนย์ด้วยซอฟต์แวร์) เพื่อกำหนดจุดอ้างอิงเริ่มต้น

การทดลองที่ 2: การจัดการ Overflow และการแปลงหน่วย

1. จุดประสงค์

เพื่อศึกษาปัญหา Overflow ของตัวนับ 16-bit และออกแบบอัลกอริทึม Wrap-around เพื่อแก้ไข และเพื่อแปลงค่าตำแหน่งสะสม (Pulses) เป็นหน่วยตำแหน่งเชิงมุม (Radians) และความเร็วเชิงมุม (rad/s)

2. สมมติฐาน

- สัญญาณ Raw Count จะแสดงพฤติกรรมคลื่นฟันเลื่อย (Sawtooth) เมื่อนับค่าเกิน 65535
- อัลกอริทึม Wrap-around สามารถตรวจจับการเกิด Overflow ของ Counts จากการคำนวณทางคณิตศาสตร์ เพื่อสร้างค่าตำแหน่งสะสมที่ต่อเนื่องได้

3. ตัวแปร

- ตัวแปรต้น: การหมุน Encoder อย่างต่อเนื่อง
- ตัวแปรตาม: Raw Count (จาก TIM3), Accumulated Count (หลัง Wrap-around), Angular Position (rad), Angular Velocity (rad/s)
- ตัวแปรควบคุม: Counter Period (65535)

4. ขั้นตอนการดำเนินงาน

1. สร้างแบบจำลอง Simulink โดยรับค่า Raw Counts จาก TIM3 (ดูรูปที่ 4 ในภาคผนวก)
2. นำค่าไปประมวลผลด้วย MATLAB Function ที่บรรจุอัลกอริทึม Wrap-around (ดูโค้ดในภาคผนวก)
3. นำค่า Accumulated Count ที่ได้ ไปคำนวณหา Angular Position และ Angular Velocity (โดยใช้ค่า PPR ที่หาได้ในการทดลองถัดไป)
4. $Position(rad) = AccumulatedCount * \left(\frac{2\pi}{Total\ Counts\ per\ Revolution} \right)$
5. $Velocity\left(\frac{rad}{s}\right) = \frac{d(Position)}{dt}$ (คำนวณโดยใช้บล็อก Derivative)
6. แสดงผลเปรียบเทียบระหว่าง Raw Count และ Accumulated Count

5. ผลการทดลอง:

- กราฟเปรียบเทียบ (กราฟที่ 4): สัญญาณ Raw Count (สีส้ม/เหลือง) มีลักษณะเป็นคลื่นฟันเลื่อยจริง โดยค่าจะวนกลับไป 0 เมื่อถึง 65535 ในขณะที่สัญญาณ Accumulated Count (สีน้ำเงิน/แดง) ที่ผ่านอัลกอริทึมแล้ว มีลักษณะเป็นเส้นที่เพิ่มขึ้นและลดลงอย่างต่อเนื่อง
- กราฟแปลงหน่วย (กราฟที่ 6, 7): ระบบสามารถแสดงผลค่าตำแหน่ง (สีเขียว/ชมพู) และความเร็ว (สีฟ้า) ในหน่วย rad และ rad/s ได้

6. **สรุปผลการทดลอง:** อัลกอริทึม Wrap-around สามารถแก้ไขปัญห Overflow/Underflow ของตัวนับ 16-bit ได้สำเร็จ ทำให้สามารถติดตามตำแหน่งสะสมได้อย่างต่อเนื่องและนำไปแปลงเป็นหน่วยทางวิศวกรรมได้
7. **อภิปรายผล:** ปัญหา Overflow เป็นข้อจำกัดทางกายภาพของรีจิสเตอร์ 16-bit การทำงานของฟังก์ชัน Wrap-around (ดูโค้ดภาคผนวก) คือการเปรียบเทียบค่าปัจจุบันกับค่าก่อนหน้า หากพบการเปลี่ยนแปลงที่ก้าวกระโดด (เช่น $\Delta \approx -65535$) ระบบจะแก้ไขค่า Δ ให้ถูกต้อง (เช่น $\Delta \approx +1$) ก่อนนำไปบวกสะสม

การทดลองที่ 3: การหาค่า PPR และความละเอียดเชิงมุม

1. จุดประสงค์

เพื่อหาค่า Pulses Per Revolution (PPR) ของ Encoder ที่ใช้ด้วยวิธีการวัดเชิงกล และคำนวณหา Angular Resolution ของระบบ

2. สมมติฐาน

ค่า PPR ที่วัดได้จากการทดลอง (โดยใช้โหมด X4) เมื่อหารด้วย 4 แล้ว จะมีค่าใกล้เคียงกับค่า PPR ที่ระบุไว้ใน Datasheet ของ Encoder (2048 PPR)

3. ตัวแปร

- **ตัวแปรต้น:** การหมุนเชิงกล (1 รอบ หรือ 360 องศา)
- **ตัวแปรตาม:** ค่า Accumulated Count ที่วัดได้
- **ตัวแปรควบคุม:** โหมดการถอดรหัส (X4)

4. ขั้นตอนการดำเนินงาน

1. สร้างจุดอ้างอิงทางกายภาพ (เช่น ใช้ Cable Tie) บนแกน Encoder
2. ทำการ Homing (ตั้งศูนย์) ที่จุดเริ่มต้น
3. หมุนแกน Encoder ด้วยมือให้ครบ 1 รอบ (360 องศา) พอดี และกลับมาที่จุดอ้างอิงเดิม
4. บันทึกค่า Accumulated Count สูงสุดที่อ่านได้จากกราฟ
5. คำนวณค่า *Measured PPR* = *Accumulated Count* / 4

6. ทำการทดลองซ้ำ 10 รอบเพื่อหาความน่าเชื่อถือของข้อมูล

5. ผลการทดลอง

- ค่าที่วัดได้ (รอบที่ 1): กราฟ (กราฟที่ 5) แสดงค่า Accumulated Count สูงสุดที่ 8225
- การคำนวณ PPR: $Measured\ PPR = \frac{8225}{4} = 2056$
- ค่าตามทฤษฎี (จาก Datasheet): PPR = 2048 ดังนั้น Total Counts (X4) ควรเป็น $2048 * 4 = 8192$
- ความคลาดเคลื่อน: $8225 - 8192 = 33\ Counts$ หรือประมาณ 0.4%
- การทดลองซ้ำ (10 รอบ): ได้ค่า 8225, 8181, 8220, 8154, 8228, 8199, 8180, 8199, 8194, 8185 ซึ่งค่าทั้งหมดใกล้เคียงกับ 8192
- การคำนวณความละเอียดเชิงมุม: $Angular\ Resolution = 360^\circ / 8192\ Counts = 0.0439$ องศาต่อ Count

6. สรุปผลการทดลอง

สามารถยืนยันได้ว่า Encoder ที่ใช้มีค่า PPR เท่ากับ 2048 ซึ่งเมื่อใช้โหมด X4 จะให้ความละเอียด 8192 Counts ต่อรอบ หรือ 0.0439 องศาต่อ Count

7. อภิปรายผล

ความคลาดเคลื่อนที่เกิดขึ้น 33 Counts (ในการทดลองรอบแรก) มีสาเหตุหลักมาจาก Human Error ในการหมุนแกนให้ครบ 360 องศาพอดี (น่าจะหมุนเกินไปเล็กน้อย) ซึ่งการทดลองซ้ำ 10 รอบ ช่วยยืนยันว่าค่าที่แท้จริงอยู่ใกล้เคียง 8192

การทดลองที่ 4: การวิเคราะห์คุณสมบัติของระบบ (Homing, Speed, Decoding)

1. จุดประสงค์

เพื่อนำฟังก์ชัน Homing ไปใช้, วิเคราะห์ผลของความเร็วต่อสัญญาณ และเปรียบเทียบโหมด X1, X2, X4

2. สมมติฐาน

- Homing: การกดปุ่มจะสามารถรีเซ็ตค่า Accumulated Count เป็น 0 ได้
- Decoding: โหมด X4 จะให้ค่า Count สูงสุด (ละเอียดสุด) เทียบกับ X1 และ X2 ในการหมุนที่เท่ากัน
- Speed: การหมุนเร็วจะทำให้สัญญาณ Velocity ที่คำนวณจาก Derivative มี Noise มากกว่าการหมุนช้า

3. ขั้นตอนการดำเนินงาน

- Homing: เพิ่ม Logic การอ่านค่าปุ่มกด B1 (PC13) เพื่อใช้รีเซ็ตค่าตัวแปร total_count ในฟังก์ชัน Wrap-around ให้เป็น 0
- Decoding: สร้างแบบจำลองที่อ่านค่าจาก TIM1 (X2), TIM3 (X4), และ TIM4 (X1) พร้อมกัน แล้วพล็อตเทียบกัน
- Speed: ใช้โมเดลที่คำนวณ Angular Velocity และทดลองหมุนช้าๆ สม่่าเสมอ เทียบกับการหมุนเร็วและกระตุก

4. ผลการทดลอง

- Homing (กราฟที่ 8): พบว่าเมื่อกดปุ่ม B1 สามารถรีเซ็ตค่า Accumulated Count และ Angular Position กลับไปเป็น 0 ได้สำเร็จ
- Decoding (กราฟที่ 10): กราฟเปรียบเทียบแสดงให้เห็นว่าในการเคลื่อนที่เดียวกัน สัญญาณ X4 (สีเหลือง) ให้ค่า Count สูงสุด ตามด้วย X2 (สีเขียว) และ X1 (สีแดง) ตามลำดับ
- Speed (กราฟที่ 11, 12):
 - หมุนช้า: สัญญาณ Velocity (กราฟที่ 11) ค่อนข้างเรียบและมี Noise ต่ำ
 - หมุนเร็ว: สัญญาณ Velocity (กราฟที่ 12) มีลักษณะเป็นหนามแหลม (Spikes) และมีการแกว่งของ Noise ที่รุนแรงกว่ามาก

5. สรุปผลการทดลอง

- ฟังก์ชัน Homing ทำงานได้ตามที่ออกแบบ ซึ่งจำเป็นสำหรับระบบที่ใช้ Encoder แบบ Relative

- โหมด X4 ให้ความละเอียดสูงสุด (4x PPR)
- ความเร็วในการหมุนที่สูงและไม่สม่ำเสมอ ส่งผลเสียต่อคุณภาพของสัญญาณ Velocity ที่คำนวณได้

6. อภิปรายผล:

- สาเหตุที่สัญญาณ Velocity มี Noise สูงเมื่อหมุนเร็ว มาจากผลการคำนวณทางคณิตศาสตร์ของบล็อก Derivative
- บล็อก Derivative นี้เป็นการคำนวณอัตราการเปลี่ยนแปลง การเปลี่ยนแปลงค่าตำแหน่ง (Count) อย่างรวดเร็ว หรือการสั่นสะเทือนทางกล (Jitter) แม้เพียงเล็กน้อย จะถูกขยาย โดยกระบวนการ Derivative ทำให้ผลลัพธ์พุ่งสูงเป็น Spikes

การเปรียบเทียบ: CUI Devices (2048 PPR) vs. BOURNS (24 PPR)

เพื่อทำความเข้าใจผลกระทบของความละเอียด (Resolution) ได้ดียิ่งขึ้น จึงมีการเปรียบเทียบ Encoder ที่ใช้ในการทดลองหลัก (CUI Devices AMT103-V) กับ Encoder อีกรุ่นหนึ่งคือ BOURNS PEC11R-4220F-N0024 ซึ่งมีคุณสมบัติที่แตกต่างกันโดยสิ้นเชิง ดัง (ตารางที่ 1)

1. การคำนวณ Total Counts และ Angular Resolution (BOURNS)

- **Total Counts (ในโหมด X4)**

จาก Datasheet, Encoder รุ่น BOURNS มีความละเอียดคงที่ 24 PPR เมื่อนำมาใช้กับโหมดการถอดรหัสแบบ X4 (Quadrature Decoding) ซึ่งนับทุกขอบของสัญญาณ A และ B จะได้จำนวนนับทั้งหมดต่อรอบ คือ:

- $Total\ Counts = PPR * 4$
- $Total\ Counts = 24 * 4 = 96\ Counts$

- **Angular Resolution**

ความละเอียดเชิงมุม (Angular Resolution) คือมุมที่เล็กที่สุดที่ Encoder สามารถตรวจจับการเปลี่ยนแปลงได้ (1 Count) ซึ่งคำนวณได้ดังนี้:

- $Angular\ Resolution = 360^\circ / Total\ Counts\ per\ Revolution$
- $Angular\ Resolution = 360^\circ / 96 = 3.75^\circ\ per\ Count$

หมายความว่า ในทุกๆ การหมุน 3.75 องศา จะเกิดการเปลี่ยนแปลงของค่า Accumulated Count เพียง 1 ครั้ง

2. การวิเคราะห์ผลกระทบต่อสัญญาณ (Position and Velocity)

ความละเอียดที่ต่ำมาก (96 Counts) นี้ ส่งผลโดยตรงต่อลักษณะของกราฟที่ได้ เมื่อเทียบกับ Encoder ความละเอียดสูง (8192 Counts) ที่ใช้ในการทดลองหลัก:

- **กราฟตำแหน่ง (Angular Position):**

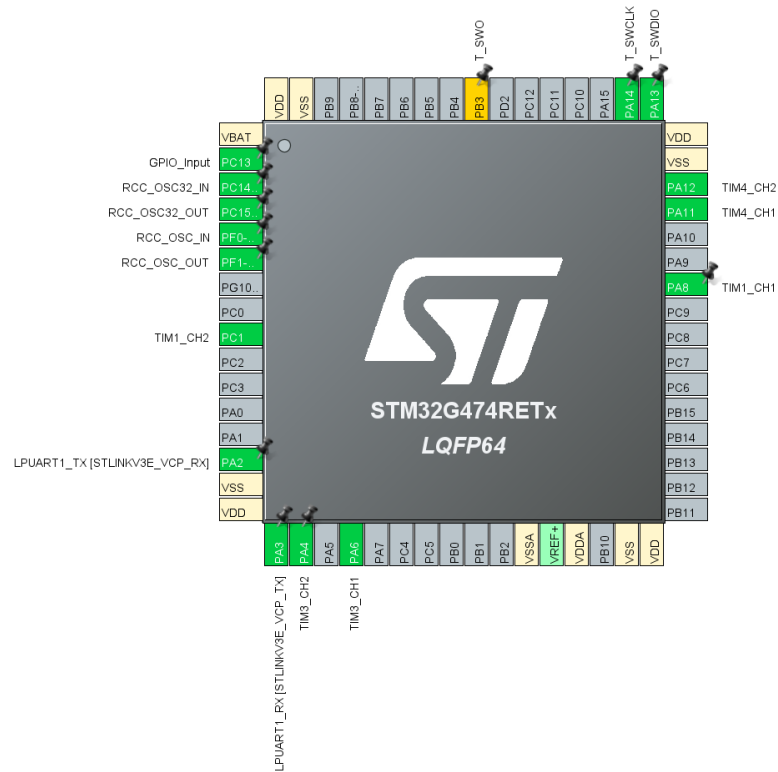
- **BOURNS (24 PPR):** กราฟมีลักษณะเป็น "ขั้นบันไดหยาบ" (Coarse Steps) อย่างชัดเจน (กราฟที่ 13) ค่าตำแหน่งจะคงที่เป็นช่วงยาว และจะเปลี่ยนแปลง (กระโดด) ก็ต่อเมื่อหมุนผ่าน 3.75 องศา
- **CUI (2048 PPR):** กราฟมีความละเอียดสูงมากจนเกือบจะดูเหมือนเส้นตรงที่ราบเรียบ (Smooth Ramp) (กราฟที่ 14)

- **กราฟความเร็ว (Angular Velocity):**

- **BOURNS (24 PPR):** เนื่องจากค่าตำแหน่ง (Position) เปลี่ยนแปลงอย่างฉับพลันทีละ 1 Count หลังจากทีค่าคงที่เป็นเวลานาน บล็อก Derivative จึงคำนวณค่าความเร็วออกมาเป็น "หนามแหลมที่กระจัดกระจาย" (Scattered Spikes) (กราฟที่ 13) ซึ่งไม่ต่อเนื่อง
- **CUI (2048 PPR):** ให้สัญญาณความเร็วที่มีความต่อเนื่องมากกว่า ทำให้สามารถวิเคราะห์ความเร็วเชิงมุมได้ดีกว่า (กราฟที่ 14)

ภาคผนวก

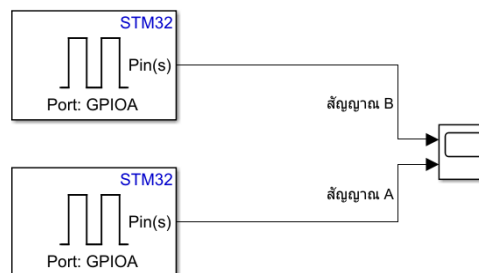
1. แผนผัง Pinout (IOC)



2. แบบจำลอง Simulink

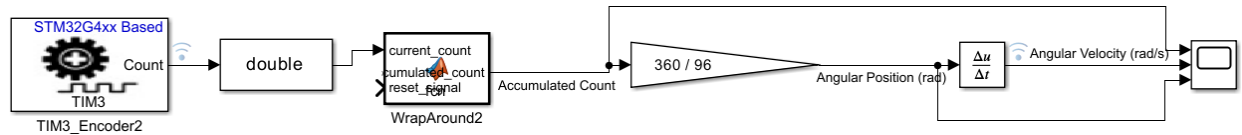
- รูปที่ 1: Simulink Model สำหรับอ่านสัญญาณดิบ (Experiment 1)

Reading Raw Signals



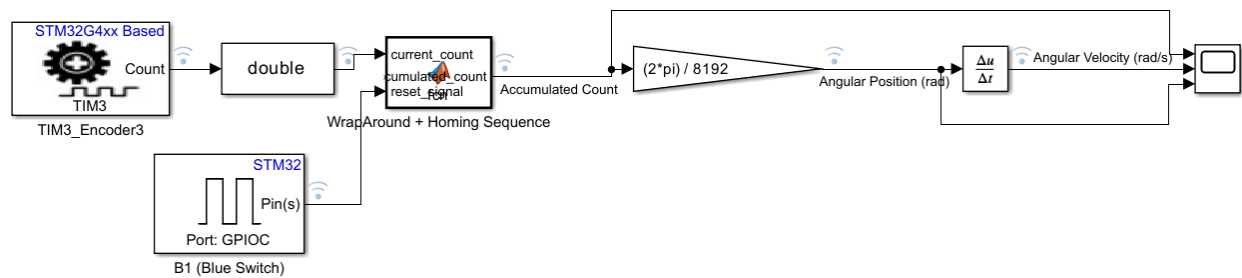
- รูปที่ 4: Simulink Model สำหรับ Overflow และ Wrap-around (Experiment 2)

Angular Position (Radians)



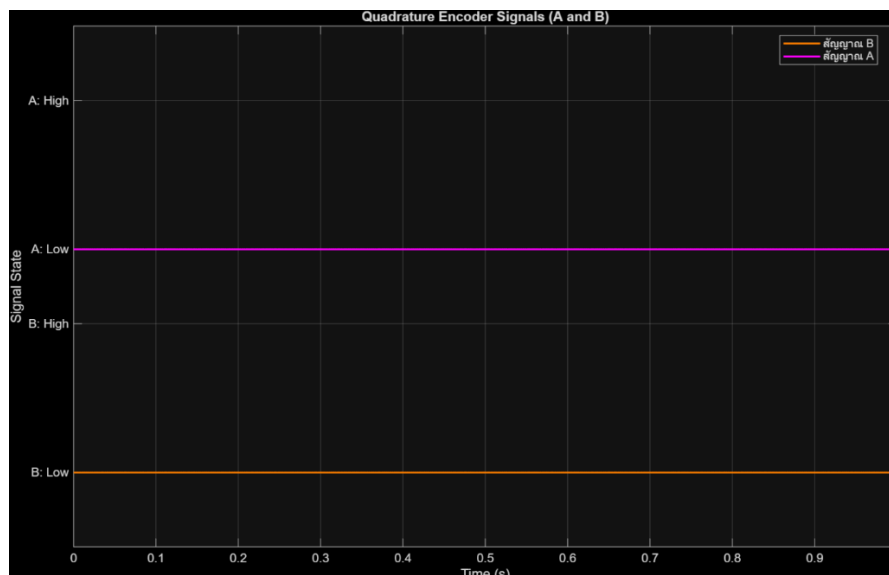
- รูปที่ 5: Simulink Model สำหรับ Homing Sequence (Experiment 4)

Homing Sequence

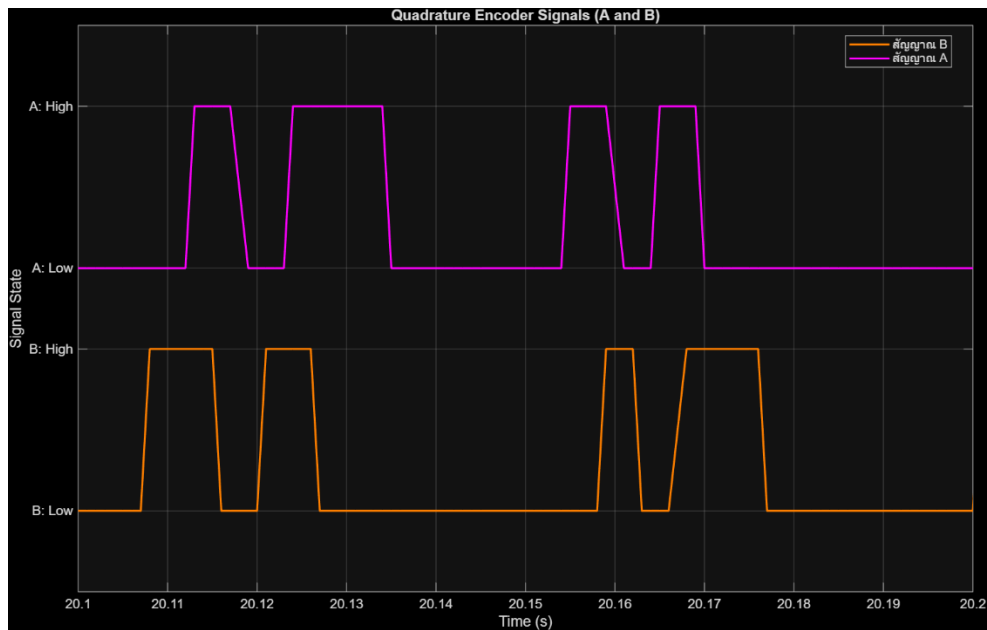


3. กราฟผลการทดลอง

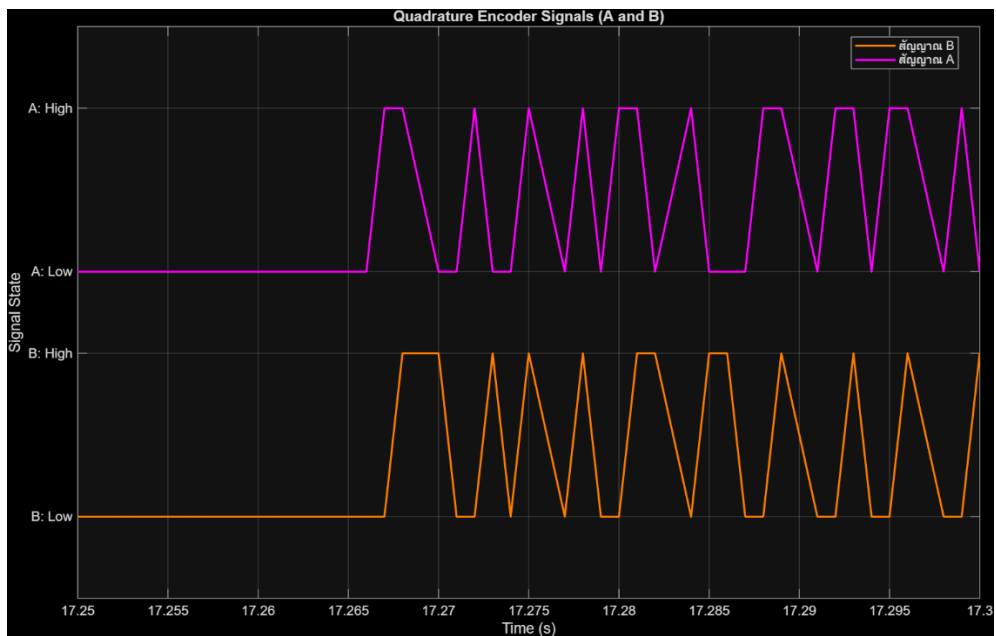
- กราฟที่ 1: สัญญาณ A และ B ขณะหยุดนิ่ง



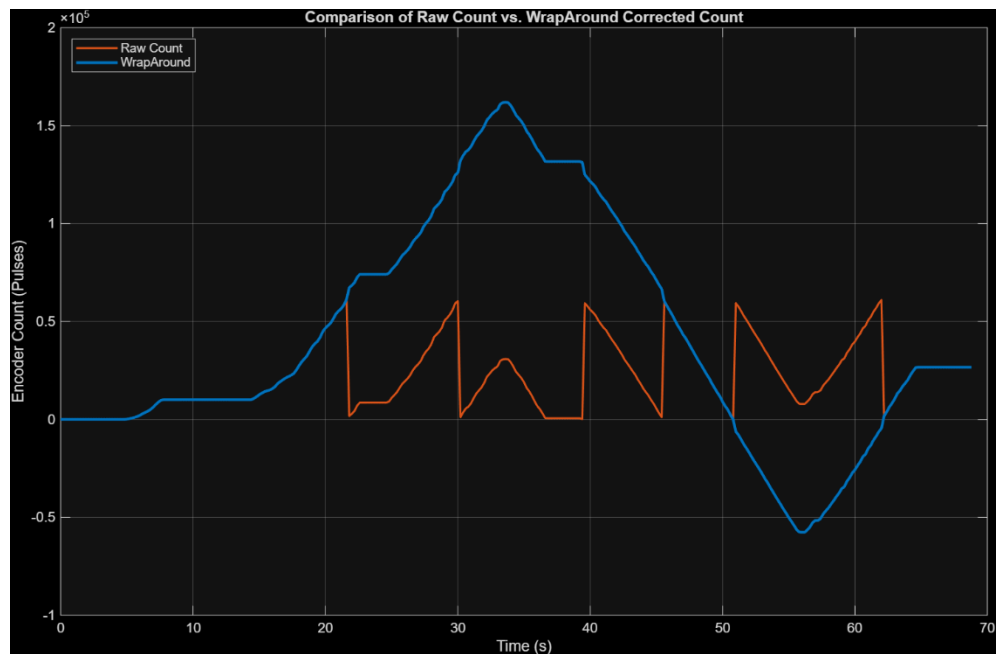
- กราฟที่ 2: สัญญาณ A และ B ขณะหมุนตามเข็มนาฬิกา (CW)



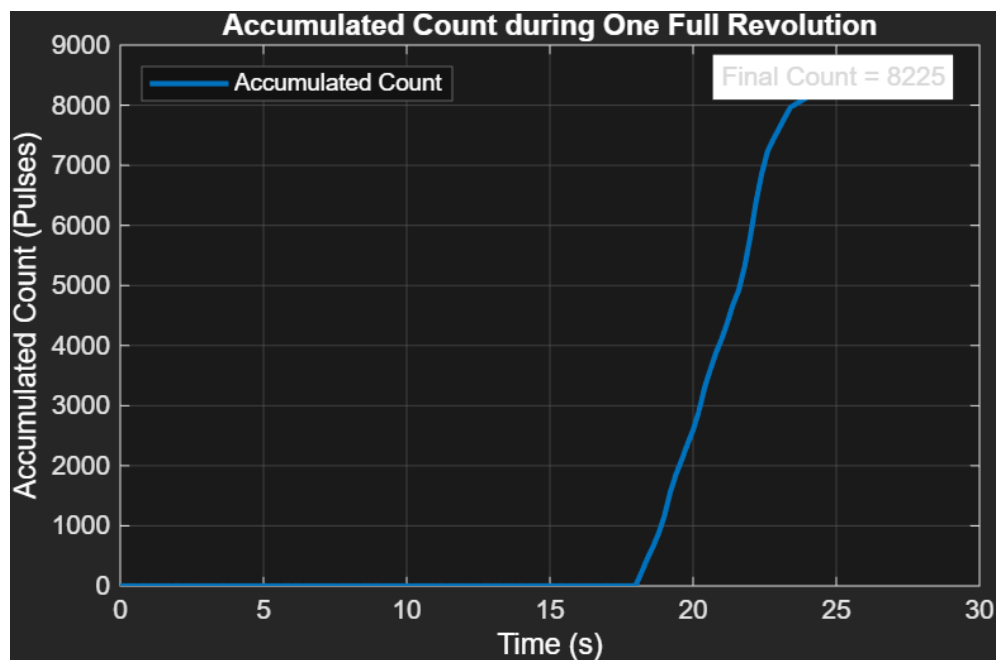
- กราฟที่ 3: สัญญาณ A และ B ขณะหมุนทวนเข็มนาฬิกา (CCW)



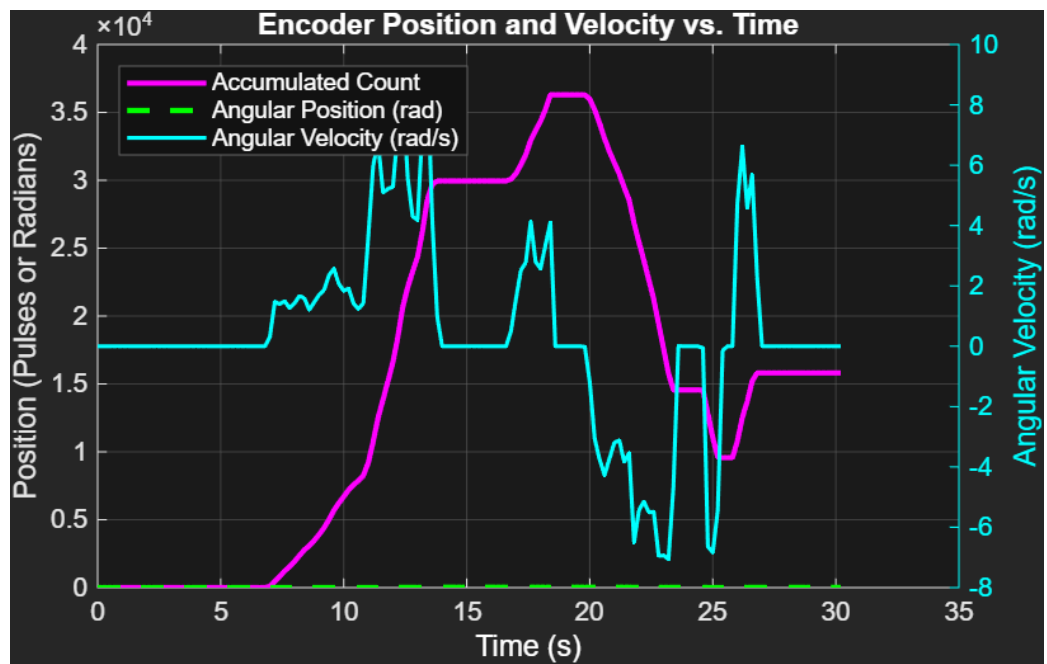
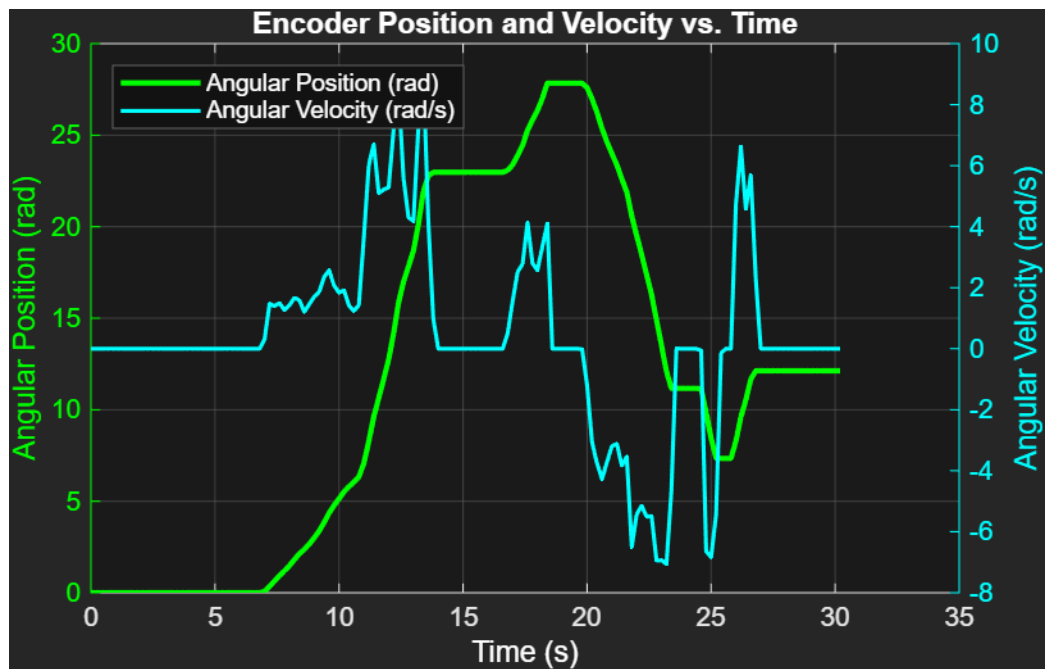
- กราฟที่ 4: กราฟเปรียบเทียบ Raw Count (ฟันเลื่อย) และ Accumulated Count (ต่อเนื่อง)



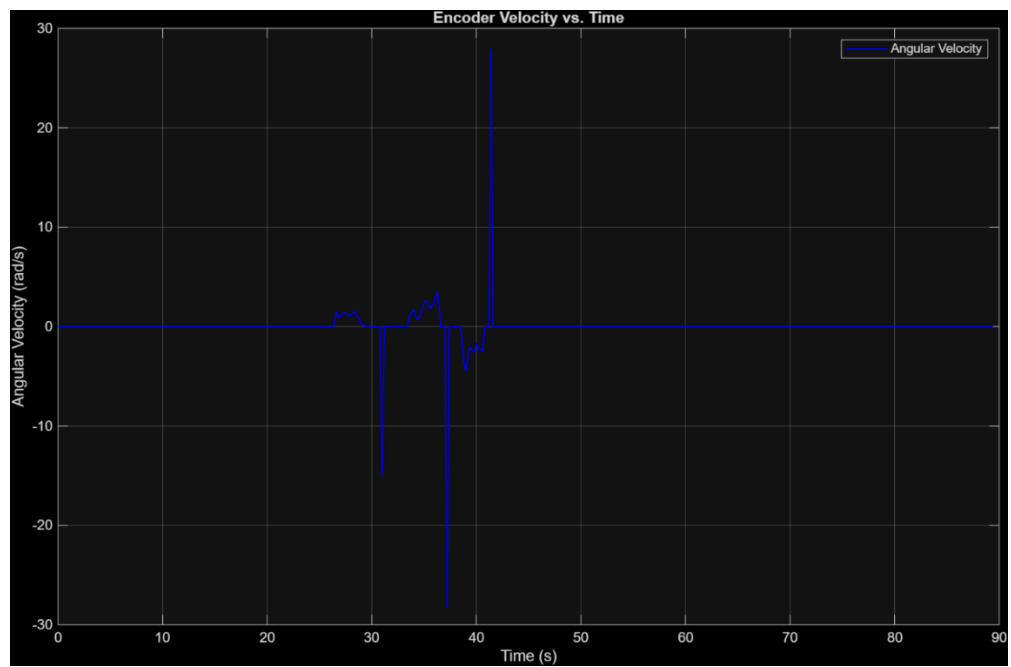
- กราฟที่ 5: กราฟ Accumulated Count จากการหมุนครบ 1 รอบ (แสดง Final Count = 8225)



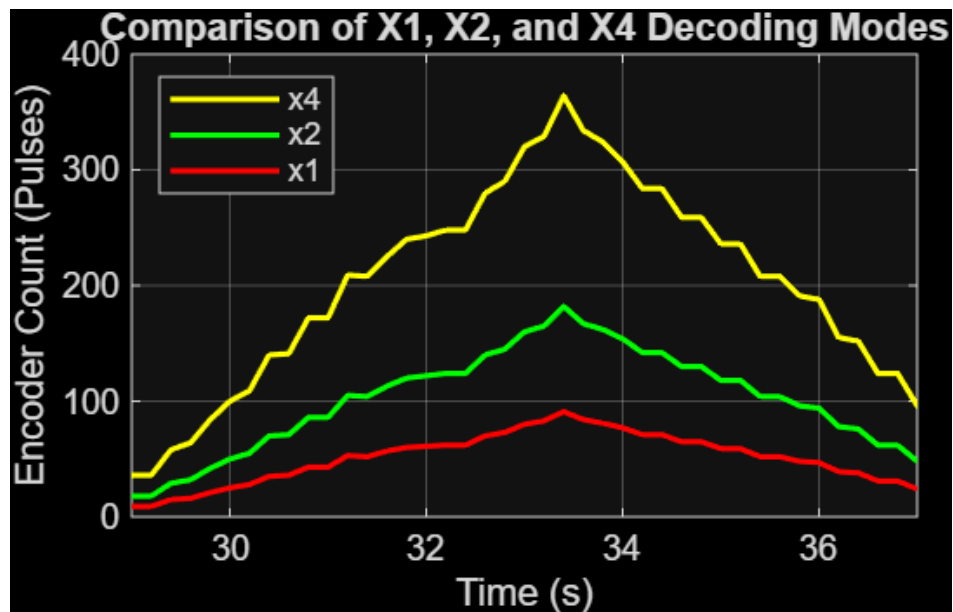
- กราฟที่ 6, 7: กราฟแสดง Position (rad) และ Velocity (rad/s)



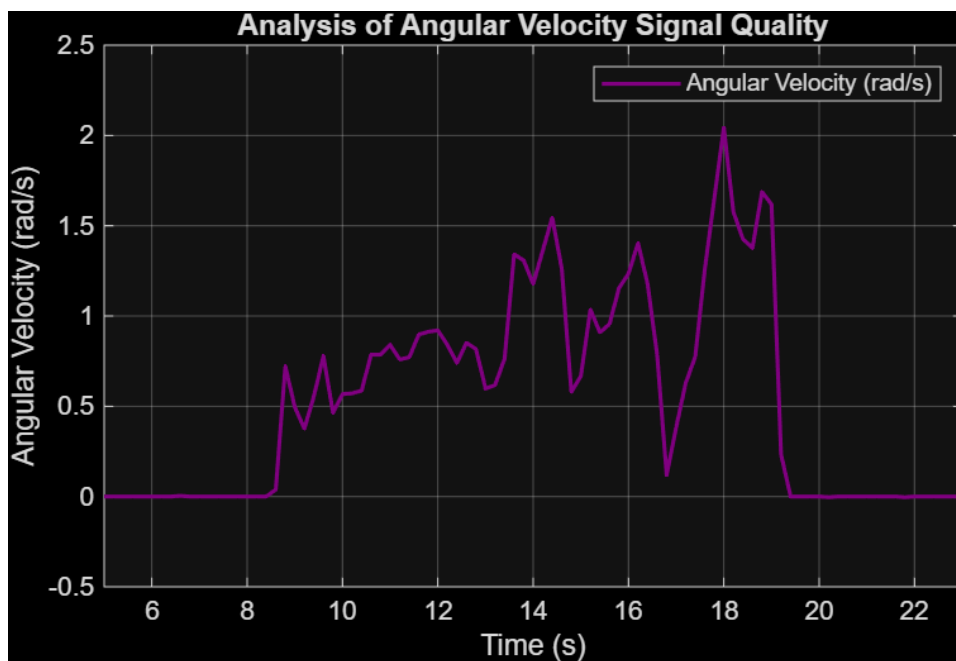
- กราฟที่ 8, 9: กราฟ Position และ Velocity หลังการทำ Homing



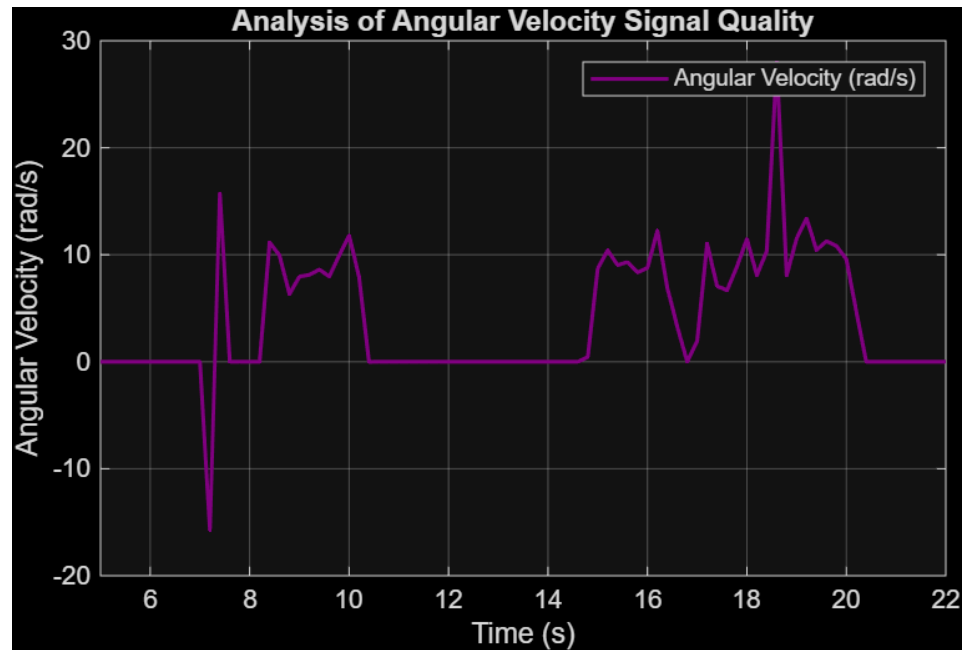
- กราฟที่ 10: กราฟเปรียบเทียบ Decoding Modes (X1, X2, X4)



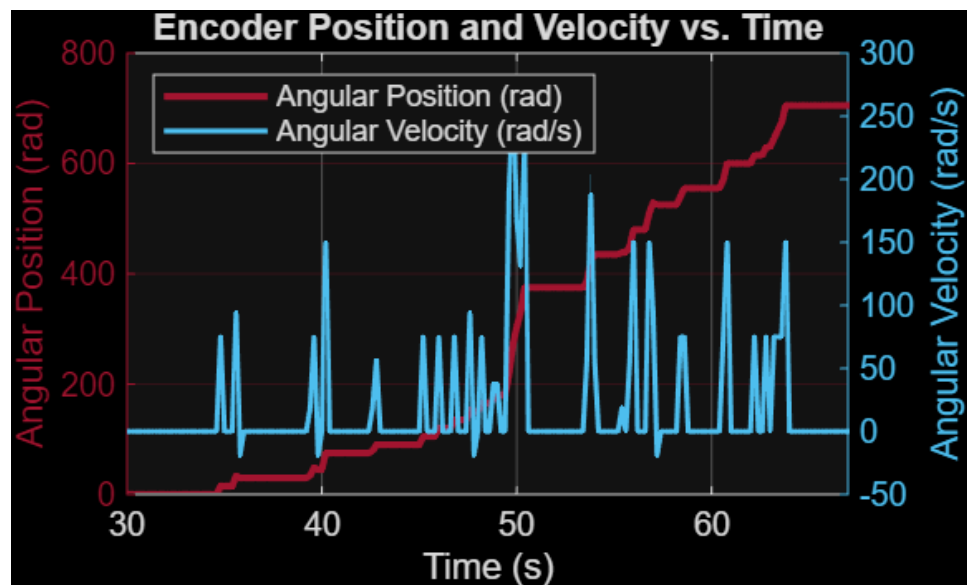
- กราฟที่ 11: สัญญาณ Velocity ขณะหมุนช้า



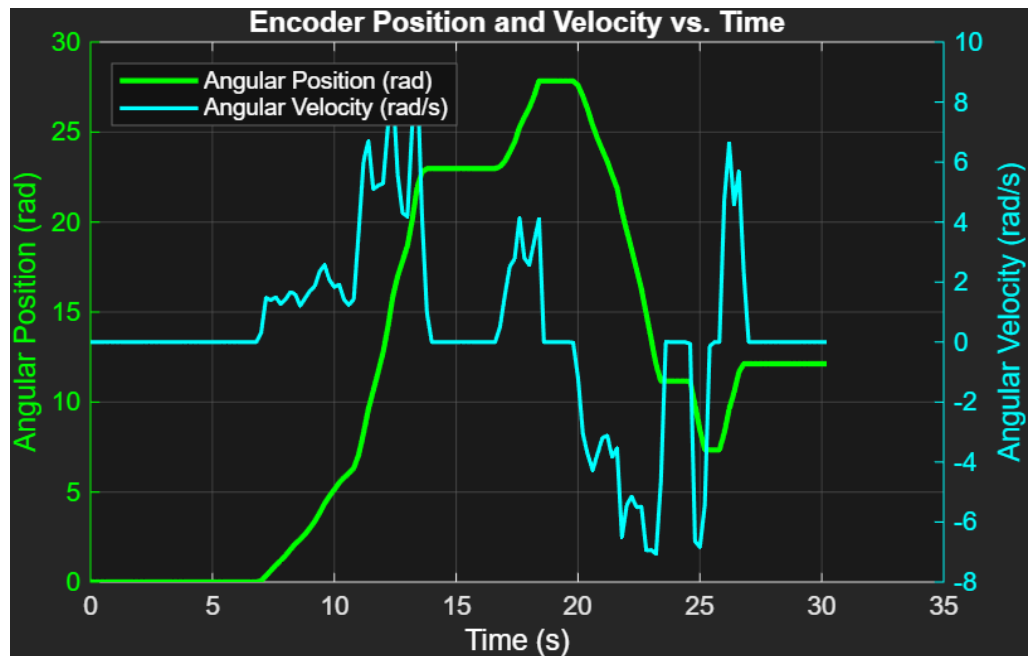
- กราฟที่ 12: สัญญาณ Velocity ขณะหมุนเร็ว (แสดง Spikes/Noise)



- กราฟที่ 13: กราฟแสดงตำแหน่งเชิงมุมและความเร็วเชิงมุมต่อเวลาของ Encoder BOURNS (24 PPR)



- กราฟที่ 14: กราฟแสดงตำแหน่งเชิงมุมและความเร็วเชิงมุมต่อเวลาของ Encoder CUI (2048 PPR)



- ตารางที่ 1: เปรียบเทียบคุณสมบัติระหว่าง Encoder CUI Devices (2048 PPR) vs. BOURNS (24 PPR)

คุณสมบัติ	CUI Devices AMT103-V (ที่ใช้ใน Lab)	BOURNS PEC11R-4220F-N0024
เทคโนโลยี	คาปาซิทีฟ (Capacitive)	เชิงกล (Mechanical)
Resolution (PPR)	48 - 2048 (ปรับค่าได้)	24 (ค่าคงที่)
ลักษณะการหมุน	หมุนเรียบ (Smooth) ต่อเนื่อง	มีขั้น (Detents) หมุนแล้วดัง "ก๊ิกๆ"
การใช้งานหลัก	ควบคุมมอเตอร์, Motion Control ความแม่นยำสูง	ปุ่มปรับเมนู (UI Navigation)
คุณภาพสัญญาณ	ยอดเยี่ยม (คมชัดและสะอาด)	ดี (อาจมี Bounce/Jitter)

4. โค้ด MATLAB Function (Wrap-around)

```
function accumulated_count = fcn(current_count)

persistent last_count; % เก็บค่า Count ของ step ที่แล้ว
persistent total_count; % เก็บค่าตำแหน่งสะสมทั้งหมด

% กำหนดค่าเริ่มต้น (ทำงานแค่ครั้งแรกรสุด)
if isempty(last_count)
    last_count = current_count;
    total_count = current_count;
end

% ค่าคงที่
COUNTER_MAX = 65535;
COUNTER_HALF = 32768; % ครึ่งหนึ่งของช่วง

% คำนวณหาค่า delta
delta = current_count - last_count;

% ตรวจสอบและแก้ไข Overflow / Underflow
if (delta > COUNTER_HALF)
    % เกิด Underflow (เช่น 1 -> 65534, delta ~ 65533)
    delta = delta - (COUNTER_MAX + 1); % แก้ไข delta ให้ติดลบ
elseif (delta < -COUNTER_HALF)
    % เกิด Overflow (เช่น 65534 -> 1, delta ~ -65533)
    delta = delta + (COUNTER_MAX + 1); % แก้ไข delta ให้เป็นบวก
end

% บวกสะสมค่า delta ที่แก้ไขแล้ว
total_count = total_count + delta;

% อัปเดตค่าล่าสุด
last_count = current_count;
```

```
% ส่งค่ากลับ  
accumulated_count = total_count;  
  
end
```

เอกสารอ้างอิง

- SCMA Co., Ltd.: Encoder (เอ็นโค้ดเดอร์) คืออะไร หลักการทำงานเป็นอย่างไร?
<https://scma.co.th/blog/post/Encoder>
- PLCX Automation: โครงสร้าง Incremental Encoder
<https://plcautomation.com/%E0%B9%82%E0%B8%84%E0%B8%A3%E0%B8%87%E0%B8%AA%E0%B8%A3%E0%B9%89%E0%B8%B2%E0%B8%87-incremental-encoder/>
- W Tech: Encoder (เอ็นโค้ดเดอร์) Omron
<http://jwtech.co.th/activity/?p=2797>
- CUI Devices AMT103-V Datasheet
https://www.alldatasheet.com/view.jsp?Searchword=Amt103-v%20datasheet&gad_source=1&gad_campaignid=163458844&gbraid=0AAAAADcdDU_DBv_eCrKaMshlpPYrxiEYYY&gclid=CjwKCAjw04HIBhB8EiwA8jGNbZ0C2Lk2ZZo45P8PaqiEFbRaIrCINnSEEtMpznuQLDXnSflH0We-BoC2AkQAvD_BwE
- BOURNS PEC11R-4220F-N0024
https://www.alldatasheet.com/view.jsp?Searchword=Pec11r&gad_source=1&gad_campaignid=170327939&gbraid=0AAAAADcdDU-D4n4uDunm6X06w_np8-EtM&gclid=CjwKCAjw04HIBhB8EiwA8jGNbQZ_rUuMBpeSued8qZeMAsSxknIW6ZUEy1jYJ76H1owhEfb3ILGaMxoCGSIQAvD_BwE